Java Arrays & Introduction to Strings

Java Arrays & Introduction to Strings

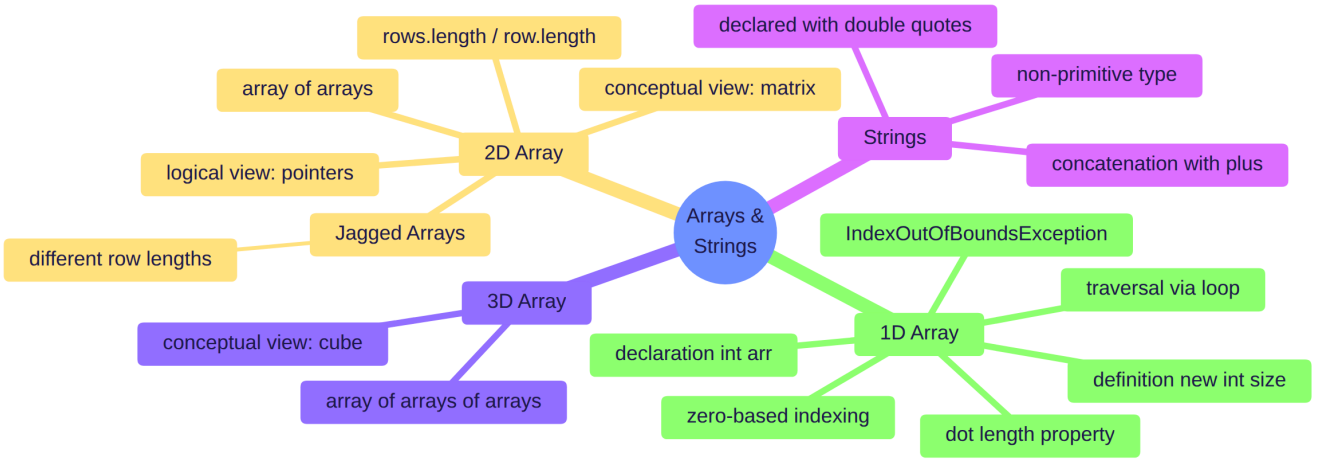
Declaration, Initialization, Traversal, Multi-Dimensional Arrays, and Strings

1. Why Arrays Exist

Imagine storing the roll numbers of 100 students. Without arrays, you'd need 100 separate variables:

```
int rollNumber1 = 101;
int rollNumber2 = 102;
int rollNumber3 = 103;
// ...and 97 more
```

This clearly doesn't scale. Instead of allocating memory for each value separately and scattered across memory, an **array** allocates one large, **contiguous** block of memory upfront and divides it into equal-sized sections — one per value.



> **Definition:** An array is a **collection of a single data type**, stored in contiguous memory, accessed using index numbers.

💡 Easy Hinglish Explanation

> "Array kuch nahi hota — bas ek badi continuous memory hoti hai jisko chhote-chhote equal parts mein baant diya jaata hai. Har part ek value store karta hai, aur har part ka apna ek number (index) hota hai jisse hum use access karte hain."

---

2. Declaring and Defining a 1D Array

Just like variables, arrays have a separate **declaration** step and **definition** step.

Declaration

```
int[] rollNums;
```

This tells the compiler: *"I intend to create a collection where every element is an* `int` *."* No memory is allocated yet.

📌 **Important:** The square brackets `[]` can legally go in **two places** — either after the data type (`int[] rollNums`) or after the array's name (`int rollNums[]`). Both compile identically. The first form is Java's preferred style; the second exists for legacy compatibility with C/C++, where arrays are declared that way.

Definition

```
int[] rollNums = new int[3];
```

Part	Meaning
`int[]`	Data type of the array
`rollNums`	Identifier / name of the array
`new`	Special keyword — allocates memory in the **heap**
`int[3]`	Data type + **size** of the array (3 elements)

Declaration and definition can be written **together** (as above) or **separately**:

```
        int[] rollNums;
        rollNums = new int[3];
```

Why the Compiler Needs the Size Upfront

The compiler must know exactly how much memory to reserve *before* allocating it. If you want to store 3 integers, and each `int` needs 32 bits, the compiler reserves **3 × 32-bit containers**, placed **contiguously** (back-to-back) in memory — for example, if the first container's address is `1001`, the next is `1033` (1001 + 32 bits), and so on.

```
int x = 4;              int[] arr = new int[3];

  x                        arr
┌─────┐              ┌──────┬──────┬──────┐
│  4  │              │      │      │      │
└─────┘              └──────┴──────┴──────┘
 32 bits               32     32     32  bits
                      (contiguous memory)
```

---

3. Zero-Based Indexing and Accessing Elements

Every element in an array has a position number called an **index** — and Java arrays **always start indexing at 0**, not 1.

```
        int[] rollNums = new int[3];
        rollNums[0] = 101;
        rollNums[1] = 102;
        rollNums[2] = 103;
```

Index	0	1	2
Value	101	102	103

Reading a value:

```
        System.out.println(rollNums[0]);   // 101
        System.out.println(rollNums[1]);   // 102
        System.out.println(rollNums[2]);   // 103
```

⚠ **Common Mistake:** For an array of size `n`, valid indices only run from `0` to `n - 1`. Beginners often assume the last valid index equals the size itself — it doesn't. An array of size 3 has indices `0, 1, 2` only; there is no index `3`.

---

4. Traversing Arrays with Loops

Manually writing `rollNums[0] = ...`, `rollNums[1] = ...` for every single element doesn't scale any better than separate variables did. Loops solve this — combined with the array's `.length` property.

Filling an Array Using a Loop

```
int[] rollNums = new int[3];
int x = 101;
for (int i = 0; i < rollNums.length; i++) {
    rollNums[i] = x;
    x++;
}
```

Each pass through the loop stores the current value of `x` into `rollNums[i]`, then increments both `i` (via the loop) and `x`.

Printing an Array Using a Loop

```
for (int i = 0; i < rollNums.length; i++) {
    System.out.println(rollNums[i]);
}
```

The `.length` Property

> **Definition:** `.length` is a built-in property that returns an array's total size — the number of elements it can hold.

```
System.out.println(rollNums.length);   // 3
```

✅ **Best Practice:** Always write loop conditions as `i < array.length` instead of hardcoding the number (`i < 3`). If the array's size ever changes, hardcoded loops silently become wrong or throw errors — `.length` keeps the loop correct automatically, no matter the array's actual size.

`IndexOutOfBoundsException`

Trying to access an index that doesn't exist — like `rollNums[3]` or `rollNums[4]` on a 3-element array — doesn't fail silently. Java throws a specific runtime error:

```
IndexOutOfBoundsException
```

📌 **Important:** An **exception** is simply Java's way of telling you *"something went wrong while running this program."* This particular one means you tried to fetch an element outside the array's valid index range. Exceptions as a full topic (types, handling, etc.) come later — for now, just recognize that this specific exception means an out-of-range index was used.

---

5. Array Initializer Shorthand

Instead of declaring an array and filling it index-by-index, Java allows directly listing all values at once:

```
int[] rollNums = {4, 5, 6};
```

This single line declares, defines, allocates the exact size needed (3, inferred from the number of values), and fills it — all at once.

---

6. Multi-Dimensional Arrays

From 1D to 2D: Why?

A 1D array stores a single row of values — e.g., one student's roll number. But what if you need **student marks across multiple subjects**, for multiple students? That's inherently **tabular** data — rows (students) and columns (subjects) — and a single-dimension array can't represent that shape.

> **Definition:** A 2D array is an **array of arrays**. Each element of the outer array is itself an entire array.

Declaring and Defining a 2D Array

```
int[][] marks = new int[3][3];
```

Part	Meaning
First `[3]`	Number of **rows**
Second `[3]`	Number of **columns**

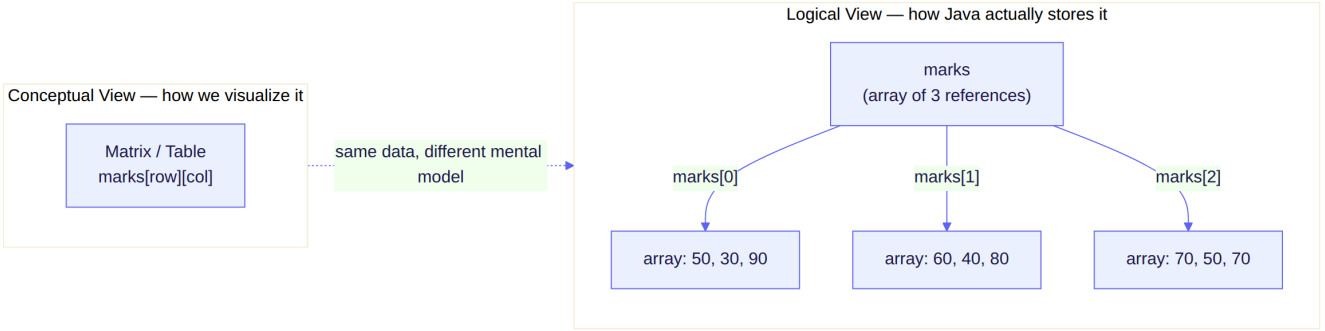
Setting individual values:

```
marks[0][0] = 50;   // Row 0, Column 0
marks[0][1] = 30;   // Row 0, Column 1
marks[0][2] = 90;   // Row 0, Column 2
marks[1][0] = 60;
// ...and so on
```

Or, using the initializer shorthand:

```
int[][] marks = {
    {50, 30, 90},
    {60, 40, 80},
    {70, 50, 70}
};
```

Two Ways to Picture a 2D Array



⭐ Exam Tip — this distinction matters a lot:

- **Conceptual view:** We *imagine* a 2D array as a neat grid/matrix — rows and columns, like a spreadsheet. This is how humans visualize the data.

- **Logical (actual) view:** Java does **not** store it as a grid internally. It stores **one array of references**, where each reference points to a **separate**, independently-allocated array. `marks` is an array of 3 elements, and `marks[0]`, `marks[1]`, `marks[2]` are each their own array of integers.

📌 **Important:** This is *why* jagged arrays (below) are even possible — since each row is a genuinely separate array object, nothing forces them to be the same length.

💡 **Easy Hinglish Explanation**

> "Hum jab 2D array ko sochte hain, to ek table jaisa imagine karte hain — rows aur columns. Lekin computer ke andar aisa nahi hota. Computer ke andar sirf ek array hota hai jiska har element khud ek alag array hai — jaise ek array doosre chhote arrays ko point kar raha ho."

Traversing a 2D Array with Nested Loops

```
        for (int row = 0; row < marks.length; row++) {
            for (int col = 0; col < marks[row].length; col++) {
                System.out.print(marks[row][col] + " ");
            }
            System.out.println();
        }
```

Two different `.length` calls, two different meanings:

Expression	Means
`marks.length`	Number of **rows** (size of the outer array)
`marks[row].length`	Number of **columns in that specific row** (size of that row's own array)

⚠ **Common Mistake:** Writing `marks.length` in *both* the outer and inner loop conditions works fine **only** when every row happens to be the same length. It silently produces wrong results (or an exception) the moment rows have different lengths — which is exactly the jagged-array scenario below. Always use `marks[row].length` for the inner loop's bound, not a repeated `marks.length`.

---

7. Jagged (Ragged) Arrays — Rows of Different Lengths

Since each row of a 2D array is its own independent array, there's no requirement that they all be the same size. An array where rows have **different lengths** is called a **jagged array**.

Real-world motivation: Different students may take a different number of subjects — one student might have 2 subject scores, another might have 4.

Declaring a Jagged Array

```
        int[][] marks = new int[3][];   // fix the number of rows only — 3 rows, columns unspecified

        marks[0] = new int[2];   // row 0 holds 2 elements
        marks[1] = new int[3];   // row 1 holds 3 elements
        marks[2] = new int[4];   // row 2 holds 4 elements
```

📌 **Important:** When declaring a 2D array, the **row count is mandatory**, but the **column count is optional** — you can leave it out entirely (`new int[3][]`) and assign each row's own array separately afterward, each with its own

independent size.

Filling and Printing a Jagged Array

```
        marks[0][0] = 43;
        marks[0][1] = 33;

        marks[1][0] = 35;
        marks[1][1] = 56;
        marks[1][2] = 87;

        marks[2][0] = 76;
        marks[2][1] = 69;
        marks[2][2] = 54;
        marks[2][3] = 65;

        for (int row = 0; row < 3; row++) {
            for (int col = 0; col < marks[row].length; col++) {
                System.out.print(marks[row][col] + " ");
            }
            System.out.println();
        }
```

Output:

```
43 33
35 56 87
76 69 54 65
```

⭐ **Exam Tip:** This is *precisely* why the inner loop must use `marks[row].length` and not a fixed number or `marks.length` — each row genuinely has a different size, and hardcoding any single number would either miss elements or throw `IndexOutOfBoundsException`.

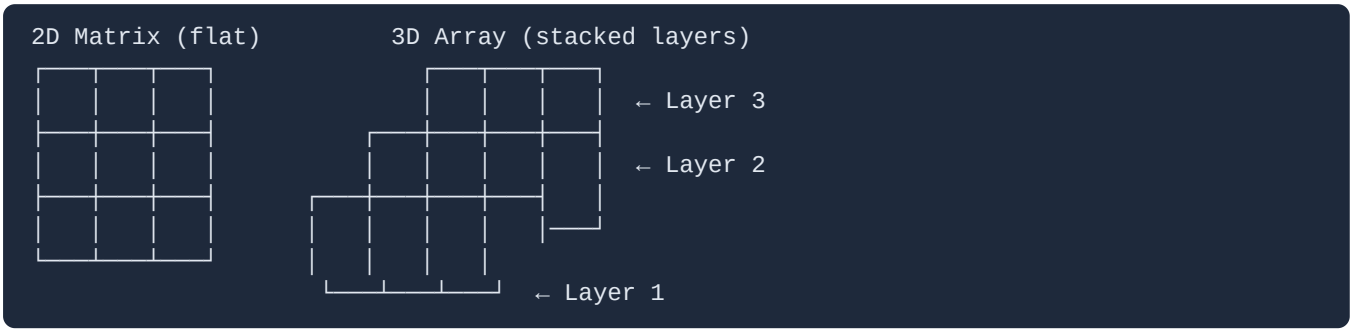
---

8. Three-Dimensional (and Beyond) Arrays

The same "array of arrays" idea extends indefinitely. A **3D array** is an array whose elements are themselves 2D arrays (which, in turn, contain 1D arrays):

```
        int[][][] arr = new int[3][3][3];
```

Conceptual visualization: If a 2D array is a flat **matrix** (a square, rows × columns), a 3D array is a **cube** — multiple 2D matrices **stacked** on top of one another as layers.



📌 **Important:** There's no theoretical limit — arrays can go to 4D, 5D, or N-dimensions. In practice, real-world code rarely goes beyond 2D or 3D, since anything higher becomes extremely difficult to reason about.

9. Introduction to Strings

Until now, every data type covered has been **primitive** (`byte`, `short`, `int`, `long`, `float`, `double`, `char`, `boolean`). Arrays are the first **non-primitive** type introduced. **String** is another.

> **Definition:** A `String` is a sequence of characters — essentially any line of text — and unlike primitives, it's a **non-primitive (object) type** in Java.

Declaring Strings

```
String firstName = "Rounak";
String lastName = "Ansari";
```

📌 **Important:** String literals are always written in **double quotes**, unlike `char` literals, which use **single** quotes (`char c = 'a';`). This distinction matters — Java uses quote type to tell primitives and strings apart at the syntax level.

⚠ **Common Mistake — from the course files:** Never name your own class `String`. Since `String` is already a built-in Java type, naming a custom class the same thing creates a naming conflict that leads to confusing errors.

String Concatenation

The `+` operator, when used on strings, doesn't perform arithmetic — it **concatenates** (joins) them together:

```
String firstName = "Rounak";
String lastName = "Ansari";
String fullName = firstName + " " + lastName;

System.out.println(fullName);   // Rounak Ansari
```

⚠ **Common Mistake:** Writing `firstName + lastName` without a space string in between concatenates with **zero** separation — producing `RounakAnsari` instead of `Rounak Ansari`. A plain `" "` (a single space) is itself a completely valid string, and is exactly what's needed to fix this.

💡 **Easy Hinglish Explanation**

> "Jab hum numbers pe `+` lagate hain, to wo addition karta hai. Lekin jab `+` strings pe lagate hain, to wo add nahi karta — sirf dono strings ko jod (concatenate) deta hai. Isliye `firstName + lastName` likhne se beech mein koi gap nahi aata, jab tak khud ek space wali string beech mein na daalo."

📌 **Important:** `String` is introduced only briefly here as a preview. Its full behavior — internal representation, immutability, and built-in methods — is covered in depth in a later, dedicated lecture, once object-oriented programming concepts are available to explain it properly.

⭐ One-Page Revision

Core Definitions

- **Array** = a collection of a single data type, stored in contiguous memory
- **Index** = the position number of an element; Java arrays are **zero-based**
- `.length` = built-in property returning an array's total size

- **2D array** = an array of arrays; conceptually a matrix, but logically a set of independently-allocated row arrays
- **Jagged array** = a 2D array whose rows have different lengths
- **3D array** = an array of 2D arrays; conceptually a stack of matrices (a cube)
- **String** = a non-primitive type representing a sequence of characters, written in double quotes

Key Syntax

```
        int[] arr = new int[5];              // 1D array, size 5
        int[] arr = {1, 2, 3};               // shorthand initializer
        int[][] marks = new int[3][3];       // 2D array, fixed 3x3
        int[][] marks = new int[3][];        // jagged — rows only, columns set
individually
        marks[0] = new int[2];               // row 0 has its own independent size
        String name = "Aditya";              // String declaration
        String full = first + " " + last;    // concatenation
```

Key Rules

- Valid indices for a size-`n` array run from `0` to `n - 1` — never `n` itself
- Accessing an invalid index throws `IndexOutOfBoundsException`
- `marks.length` = number of rows; `marks[row].length` = number of columns in that specific row
- For jagged arrays, always use `marks[row].length` in the inner loop — never a hardcoded number or `marks.length`
- String literals use double quotes; `char` literals use single quotes
- `+` on strings concatenates; it does not add numerically

📝 Practice Questions

Basic

1. What is the difference between declaring and defining an array?
2. Why does Java array indexing start at 0 instead of 1?
3. What does the `.length` property return?

Medium 4. Given `int[] arr = new int[5];`, what is the valid range of indices, and what happens if you access `arr[5]` ? 5. Write the code to declare a jagged 2D array with 3 rows of sizes 2, 4, and 1 respectively. 6. Why must the inner loop of a jagged array traversal use `marks[row].length` instead of `marks.length` ?

Interview 7. Explain the difference between the *conceptual* and *logical* representation of a 2D array in Java. 8. Why is a 2D array described as an "array of arrays" rather than a true grid in memory? 9. What happens internally when you concatenate two strings using `+` ? Why is this different from adding two integers with `+` ? 10. Describe how a 3D array can be conceptually visualized as a cube made of 2D matrices.

🔥 Quick FAQ

Can an array store mixed data types? No — a single array can only hold one data type (its declared type). Mixed-type collections require different, non-primitive structures covered later.

Is `int[] arr` different from `int arr[]` ? No — both are valid, identical declarations. The first is Java's preferred modern style; the second exists for legacy C/C++-style compatibility.

Do I need to specify both dimensions of a 2D array upfront? No — the row count is mandatory, but you can leave columns unspecified (`new int[3][]`) and assign each row's array separately, which is exactly how jagged arrays are

built.

Is `String` a primitive type? No — it's non-primitive (an object type), which is why it's introduced alongside arrays rather than with `int`, `char`, `boolean`, etc.